ARTI 403 – Image Processing

Lab 6 — Filtering in the Frequency Domain

Student name: Ahmed Albuainain

Student ID: 2240006128

Section: 7MA1

Date: 19 October 2026

Session Outcome

Outcome #1: Write a program that implements fundamental image processing algorithms.

Contents

Part	Description
Part 0	Environment check
Part 1	What the frequency domain is — the 2-D FFT and its spectrum
Part 2	The Laplacian filter in the frequency domain (the manual's Task 1)
Part 3	The convolution theorem — why multiplying frequencies filters an image
Task 1	Apply the Sobel filters in the frequency domain

Part 0 — Environment check

No image files are needed: the `cameraman` image comes directly from `skimage.data`.

In [1]:
```
import sys
import numpy as np
import matplotlib
import matplotlib.pyplot as plt
import skimage as ski
from skimage import data
from skimage.transform import resize
from numpy.fft import fft2, ifft2, fftshift, ifftshift

print("Python      :", sys.version.split()[0])
print("NumPy       :", np.__version__)
print("scikit-image:", ski.__version__)
print("Matplotlib  :", matplotlib.__version__)

%matplotlib inline
```

```
Python      : 3.13.16
NumPy       : 2.5.3
scikit-image: 0.26.0
Matplotlib  : 3.11.2
```

In [2]:
```
import os
os.makedirs("images", exist_ok=True)
os.makedirs("output", exist_ok=True)

def show(images, titles, cols=3, size=4.4, suptitle=None):
    n = len(images)
    rows = (n + cols - 1) // cols
    fig, axes = plt.subplots(rows, cols, figsize=(size * cols, size * rows))
    axes = np.atleast_1d(axes).ravel()
    for ax, im, t in zip(axes, images, titles):
        ax.imshow(im, cmap='gray')
        ax.set_title(t, fontsize=11)
        ax.axis('off')
    for ax in axes[n:]:
        ax.axis('off')
    if suptitle:
        fig.suptitle(suptitle, fontsize=14)
    plt.tight_layout()
    plt.show()
```

Part 1 — What the frequency domain is

Everything so far has worked on the image as a **grid of pixel values**. The Fourier transform describes the same image a completely different way: as a **sum of waves**. Each wave has a direction, a spatial frequency (how fast it oscillates) and an amplitude, and adding them all back together reconstructs the original image exactly.

The usefulness is that image content separates by frequency:

Frequency	What it corresponds to
Low	large smooth regions, overall brightness, gradual shading
High	edges, fine texture, noise — anything that changes quickly

So "blur" and "sharpen", which in Lab 5 meant sliding a kernel over the pixels, become something much simpler here: **keep some frequencies and suppress others**.

1.1 Loading the image

`resize` is applied as the manual suggests, which also converts the image to **float in the range 0–1** — not `uint8`. Everything downstream inherits that range.

In [3]:
```
# Load cameraman image (already grayscale)
image = data.camera()
print("as loaded :", image.shape, image.dtype, image.min(), "-", image.max())

image = resize(image, (256, 256))   # Resize for faster FFT (optional)
print("after resize:", image.shape, image.dtype,
      round(float(image.min()), 3), "-", round(float(image.max()), 3))

# Get image size
M, N = image.shape
print("M, N =", M, N)
```

```
as loaded : (512, 512) uint8 0 - 255
after resize: (256, 256) float64 0.008 - 0.999
M, N = 256 256
```

1.2 The 2-D FFT and its spectrum

`fft2` returns a **complex** array of the same shape as the image. Each entry holds the amplitude and the phase of one frequency component.

Two things are needed to look at it:

- **`np.abs`** — the magnitude, since the values are complex.
- **`np.log(1 + ...)`** — the magnitudes span an enormous range (the DC term alone is tens of thousands of times larger than the rest), so without a log the display is one white dot on black.

`fftshift` is for display only. `fft2` places the zero frequency at the *corner* of the array, not the middle. `fftshift` moves it to the centre so the spectrum is readable; it changes nothing about the data itself.

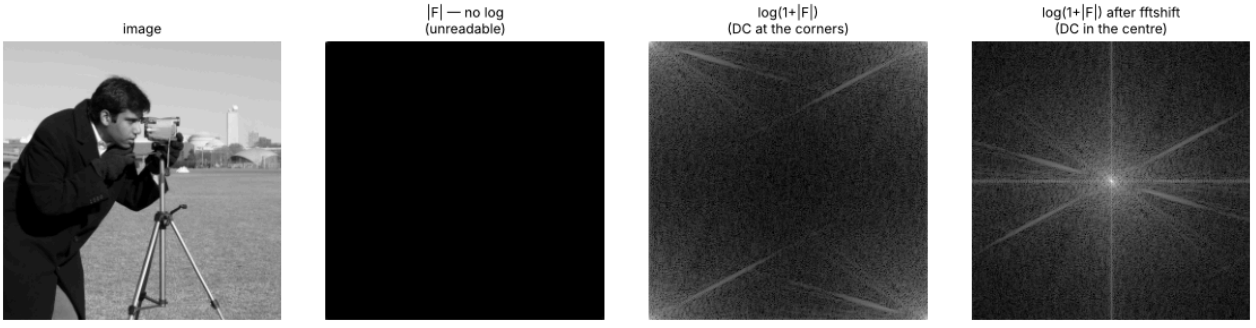
In [4]:
```
F = fft2(image)

print("F dtype:", F.dtype, "| shape:", F.shape)
print("magnitude range:", round(float(np.abs(F).min()), 2),
      "to", round(float(np.abs(F).max()), 2))
print("the largest value is at index", np.unravel_index(np.abs(F).argmax(), F.shape),
      "-> the corner, i.e. the DC (zero-frequency) term")
print("DC term / mean pixel value:", round(float(np.abs(F[0, 0]) / image.size), 4),
      "vs image mean", round(float(image.mean()), 4))

spectrum_raw = np.abs(F)
spectrum_log = np.log(1 + np.abs(F))
spectrum_centred = np.log(1 + np.abs(fftshift(F)))

show([image, spectrum_raw, spectrum_log, spectrum_centred],
     ["image", "|F| — no log\n(unreadable)", "log(1+|F|)\n(DC at the corners)",
      "log(1+|F|) after fftshift\n(DC in the centre)"],
     cols=4, size=3.7)
```

```
F dtype: complex128 | shape: (256, 256)
magnitude range: 0.01 to 33169.18
the largest value is at index (np.int64(0), np.int64(0)) -> the corner, i.e. the DC (zero-
frequency) term
DC term / mean pixel value: 0.5061 vs image mean 0.5061
```



The DC term divided by the number of pixels equals the mean pixel value exactly — a useful sanity check that the transform is behaving as expected.

In the centred spectrum, distance from the middle **is** frequency: the bright cross and the energy near the centre are the low frequencies that carry most of the image, and the faint outer regions are the high frequencies that carry the edges.

1.3 Proving that low and high frequencies carry different content

Rather than assert it, the cell below cuts the spectrum in two with a circular mask and reconstructs each half separately. Keeping only the low frequencies gives a blurred image; keeping only the high

ones gives the edges.

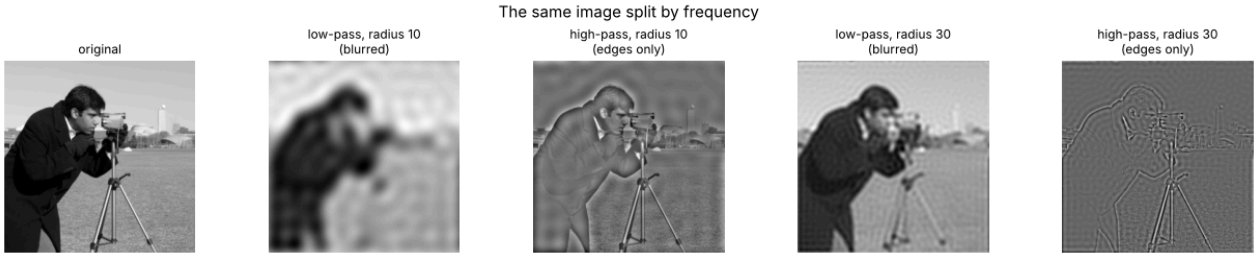
In [5]:
```
# radius of each frequency from the centre of the shifted spectrum
cy, cx = M // 2, N // 2
yy, xx = np.ogrid[:M, :N]
radius = np.sqrt((yy - cy) ** 2 + (xx - cx) ** 2)

Fs = fftshift(F)
recon = []
for r_cut in [10, 30]:
    low = ifft2(ifftshift(Fs * (radius <= r_cut))).real
    high = ifft2(ifftshift(Fs * (radius > r_cut))).real
    recon += [(low, f"low-pass, radius {r_cut}\n(blurred)"),
              (high, f"high-pass, radius {r_cut}\n(edges only)")]

show([image] + [r[0] for r in recon],
     ["original"] + [r[1] for r in recon],
     cols=5, size=3.3,
     suptitle="The same image split by frequency")

print("energy in the lowest frequencies (radius <= 30):",
      f"{100 * (np.abs(Fs)[radius <= 30] ** 2).sum() / (np.abs(Fs) ** 2).sum():.2f}%",
      "of the total — carried by", f"{100 * (radius <= 30).mean():.2f}%", "of the
coefficients")
```

```
energy in the lowest frequencies (radius <= 30): 98.97% of the total — carried by 4.30% of the
coefficients
```



Part 2 — The Laplacian filter in the frequency domain

In Lab 5 the Laplacian was a 3x3 kernel slid over the pixels. In the frequency domain it is a

formula, applied by plain multiplication:

$$H(u,v) = -4\pi^2 (u^2 + v^2)$$

The shape of this function explains what it does at a glance. It is zero at the origin — where `u = v = 0`, the DC term — and grows in magnitude with distance from it. So it **deletes the average brightness entirely** and amplifies high frequencies in proportion to how high they are. That is a high-pass filter, which is exactly what an edge detector is.

This is the same conclusion reached in Lab 5 from a different direction: there, the kernel summed to zero, so it returned nothing on flat regions. Both facts say the same thing.

In [6]:
```
# Generate frequency grid (centered)
u = np.fft.fftfreq(M).reshape(-1, 1)
v = np.fft.fftfreq(N).reshape(1, -1)

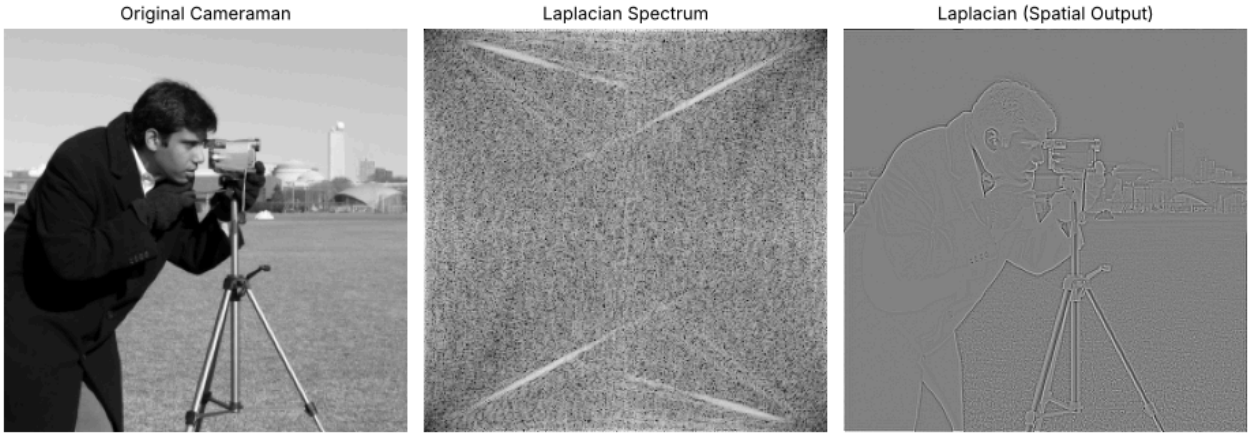
# Compute Laplacian filter in frequency domain
laplacian_filter = -4 * (np.pi ** 2) * (u**2 + v**2)

# Apply 2D FFT
F = np.fft.fft2(image)

# Apply Laplacian filter in frequency domain
F_lap = F * laplacian_filter

# Inverse FFT to get spatial result
laplacian_image = np.fft.ifft2(F_lap).real

# Plot results
plt.figure(figsize=(12, 5))
plt.subplot(1, 3, 1)
plt.imshow(image, cmap='gray')
plt.title('Original Cameraman')
plt.axis('off')
plt.subplot(1, 3, 2)
plt.imshow(np.log(1 + np.abs(F_lap)), cmap='gray')
plt.title('Laplacian Spectrum')
plt.axis('off')
plt.subplot(1, 3, 3)
plt.imshow(laplacian_image, cmap='gray')
plt.title('Laplacian (Spatial Output)')
plt.axis('off')
plt.tight_layout()
plt.show()
```



2.1 A note on the comment in the manual

The manual labels the frequency grid "centered". It is worth checking, because the word means something specific here.

In [7]:
```
print("np.fft.fftfreq(8) =", np.round(np.fft.fftfreq(8), 3))
```

```
np.fft.fftfreq(8) = [ 0.     0.125  0.25   0.375 -0.5   -0.375 -0.25  -0.125]
```

The values run `0, +, +, +, -, -, -, -`: zero first, then the positives, then the negatives. That is **not** centred — a centred grid would run from negative through zero to positive.

But this is correct, and deliberately so. `fft2` lays out its output in exactly the same order, with the zero frequency at index `[0, 0]`. Because `fftfreq` matches that layout, the multiplication `F *`

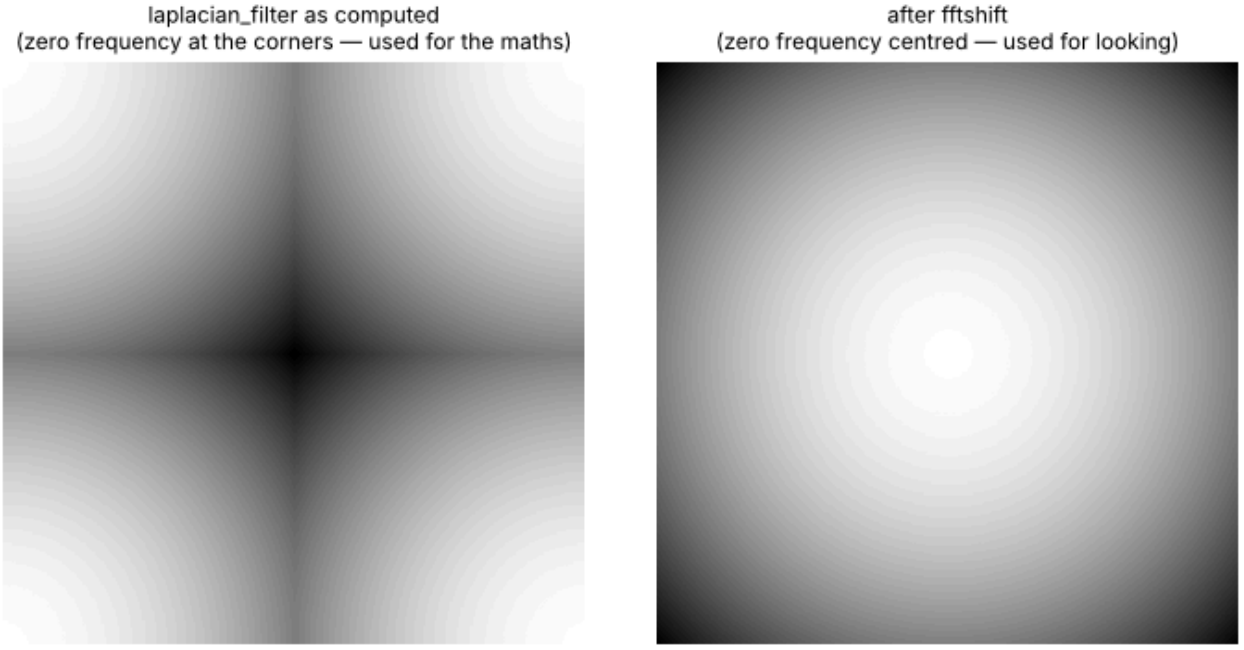
`laplacian_filter` lines up element by element. Had the grid actually been centred, every frequency would have been paired with the wrong coefficient.

The rule worth remembering: **`fftshift` is for looking, never for computing.** The cell below shows the filter in both layouts.

In [8]:
```
show([laplacian_filter, fftshift(laplacian_filter)],
     ["laplacian_filter as computed\n(zero frequency at the corners — used for the maths)",
      "after fftshift\n(zero frequency centred — used for looking)"],
     cols=2, size=4.6)

print("value at [0,0] (the DC term):", laplacian_filter[0, 0],
      " <- zero, so the average brightness is removed entirely")
print("most negative value          :", round(float(laplacian_filter.min()), 2),
      " <- at the highest frequencies")
print("output mean:", f"{laplacian_image.mean():.2e}", " <- essentially zero, as predicted")
```

```
value at [0,0] (the DC term): -0.0  <- zero, so the average brightness is removed entirely
most negative value          : -19.74  <- at the highest frequencies
output mean: 0.00e+00  <- essentially zero, as predicted
```



2.2 Comparing with the 3x3 Laplacian from Lab 5

The frequency-domain formula is the **exact continuous** Laplacian; the 3x3 kernel is a discrete approximation to it. They should agree in character but not in magnitude.

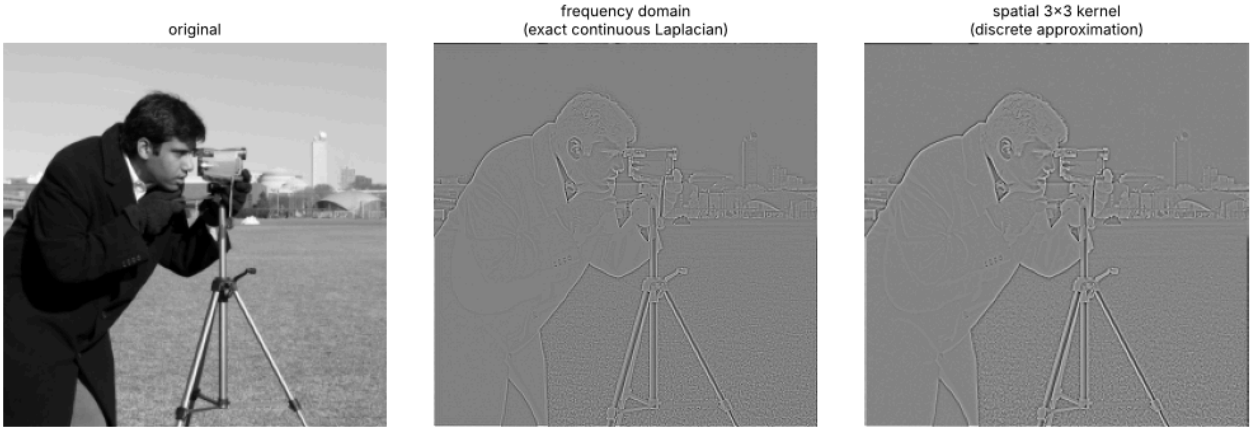
In [9]:
```
from scipy import ndimage

kernel_3x3 = np.array([[0, 1, 0], [1, -4, 1], [0, 1, 0]], dtype=float)
laplacian_spatial = ndimage.convolve(image, kernel_3x3, mode='wrap')

print("frequency-domain Laplacian:", round(float(laplacian_image.min()), 3),
      "to", round(float(laplacian_image.max()), 3))
print("3x3 spatial Laplacian     :", round(float(laplacian_spatial.min()), 3),
      "to", round(float(laplacian_spatial.max()), 3))
print("correlation between them  :",
      round(float(np.corrcoef(laplacian_image.ravel(), laplacian_spatial.ravel())[0, 1]),
4))

show([image, laplacian_image, laplacian_spatial],
     ["original", "frequency domain\n(exact continuous Laplacian)",
      "spatial 3x3 kernel\n(discrete approximation)"],
     cols=3, size=4.4)
```

```
frequency-domain Laplacian: -2.037 to 1.941
3x3 spatial Laplacian     : -1.322 to 1.182
correlation between them  : 0.9738
```



A correlation of about 0.97 with clearly different ranges is the expected result: the two respond to the same structures, but the continuous formula keeps amplifying right up to the highest representable frequency while the 3x3 kernel levels off. The frequency version is therefore the more aggressive edge detector, and the more noise-sensitive.

Part 3 — The convolution theorem

Everything in this lab rests on one result:

$f * h \;\longleftrightarrow\; F \cdot H$

Convolution in the spatial domain equals multiplication in the frequency domain. Sliding a kernel over every pixel and multiplying two arrays element by element produce the same answer.

Two consequences follow, and both matter for Task 1.

3.1 It is convolution, not correlation

Lab 5 noted that `cv2.filter2D` performs **correlation** — it does not flip the kernel. The frequency-domain product performs true **convolution**, which does. For the symmetric kernels of Lab 5 the distinction made no difference; the Sobel kernels are **antisymmetric**, so here it flips the sign.

In [10]:

```
def center_embed_kernel(kernel, shape):
    """Pad a small kernel to the image size, centred, and return its FFT."""
    padded = np.zeros(shape)
    kh, kw = kernel.shape
    ph, pw = shape
    cy, cx = ph // 2, pw // 2
    padded[cy - kh//2 : cy - kh//2 + kh, cx - kw//2 : cx - kw//2 + kw] = kernel
    return fft2(ifftshift(padded))   # Shift before FFT to place (0,0) at top-left


test_kernel = np.array([[-1, 0, 1], [-2, 0, 2], [-1, 0, 1]], dtype=float)
H_test = center_embed_kernel(test_kernel, image.shape)
freq_result = ifft2(fft2(image) * H_test).real

conv = ndimage.convolve(image, test_kernel, mode='wrap')
corr = ndimage.correlate(image, test_kernel, mode='wrap')

print("frequency product vs spatial CONVOLUTION:",
      f"{np.abs(freq_result - conv).max():.2e}", " <- identical to machine precision")
print("frequency product vs spatial CORRELATION:",
      f"{np.abs(freq_result - corr).max():.4f}", " <- completely different")
```

```
frequency product vs spatial CONVOLUTION: 1.49e-15  <- identical to machine precision
frequency product vs spatial CORRELATION: 6.6052  <- completely different
```

3.2 Why `ifftshift` is applied before the FFT

The kernel is placed in the **middle** of the padded array so it is visually centred, but the Fourier transform treats index `[0, 0]` as the origin. `ifftshift` moves the centre of the array to that corner before transforming.

Without it, the filter carries a built-in translation of half the image, and the output comes out shifted — a silent error that produces a plausible-looking but wrong image.

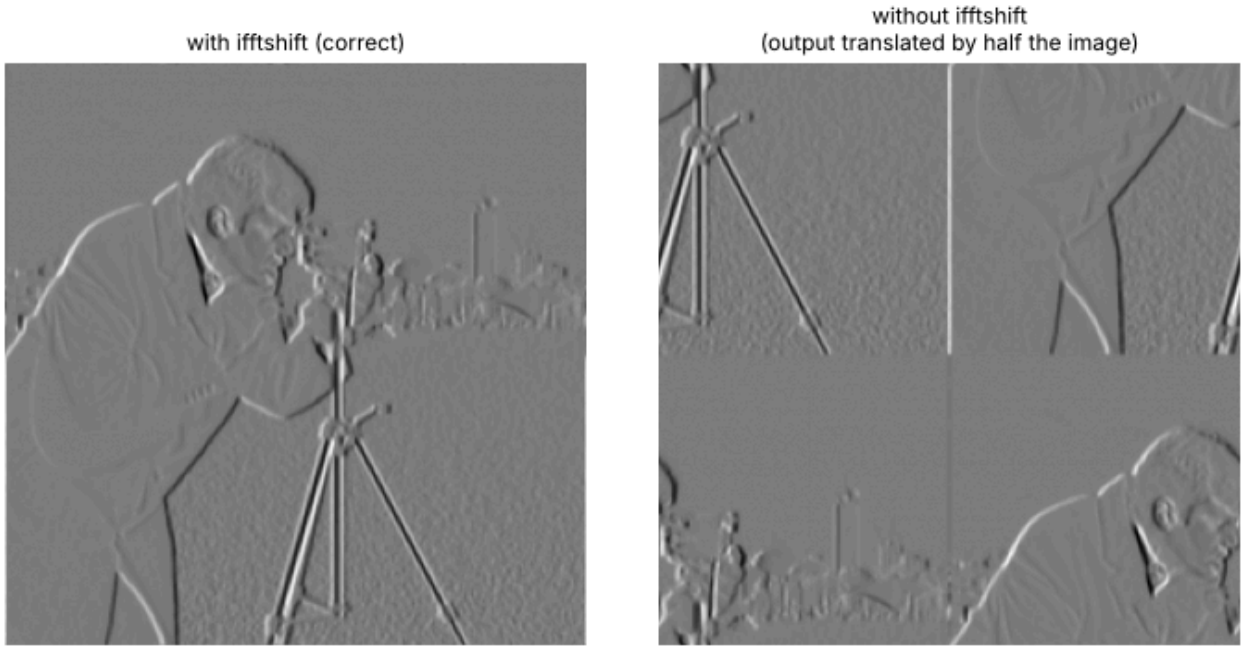
In [11]:

```
# the same kernel placed centrally, WITHOUT ifftshift
padded_wrong = np.zeros(image.shape)
padded_wrong[127:130, 127:130] = test_kernel
wrong_result = ifft2(fft2(image) * fft2(padded_wrong)).real

print("largest difference from the correct result:",
      round(float(np.abs(wrong_result - freq_result).max()), 3))
print("is it just a translation? compare with the correct result rolled by half the
image:",
      f"{np.abs(wrong_result - np.roll(freq_result, (M//2, N//2), axis=(0,
1))).max():.2e}")

show([freq_result, wrong_result],
     ["with ifftshift (correct)", "without ifftshift\n(output translated by half the
image)"],
     cols=2, size=4.6)
```

```
largest difference from the correct result: 3.797
is it just a translation? compare with the correct result rolled by half the image: 0.00e+00
```



The second print confirms the diagnosis exactly: the wrong result is the correct one shifted by half the image in each direction, to machine precision. Nothing about the filtering was wrong — only its position.

3.3 One more consequence — the FFT wraps around

The Fourier transform treats the image as **periodic**: the right edge is adjacent to the left edge, and the bottom to the top. Frequency-domain filtering is therefore *circular* convolution, and content can bleed from one edge to the opposite one.

This is why the spatial comparison above used `mode='wrap'` — any other border mode would not have matched. In practice the image is zero-padded to avoid the effect; it is left in here so the equivalence is exact.

In [12]:
```
edge_diff = np.abs(ndimage.convolve(image, test_kernel, mode='wrap')
                   - ndimage.convolve(image, test_kernel, mode='reflect'))
print("difference between 'wrap' and 'reflect' border handling:")
print("  over the whole image :", round(float(edge_diff.mean()), 5))
print("  on the border pixels :",
      round(float(np.concatenate([edge_diff[0], edge_diff[-1],
                                  edge_diff[:, 0], edge_diff[:, -1]]).mean()), 5),
      " <- the disagreement is confined to the edges")
```

```
difference between 'wrap' and 'reflect' border handling:
  over the whole image : 0.0087
  on the border pixels : 0.56044  <- the disagreement is confined to the edges
```

Task 1 — The Sobel filters in the frequency domain

The Sobel operators detect edges in one direction each: `sobel_x` responds to **vertical** edges (where intensity changes horizontally) and `sobel_y` to **horizontal** ones. Combining them gives the gradient magnitude, which responds to edges in any direction.

Following the structure given in the hints.

In [13]:
```
# Load and resize image  (already done in Part 1: `image`, M x N)

# Sobel filters
sobel_x = np.array([[-1, 0, 1],
                    [-2, 0, 2],
                    [-1, 0, 1]])

sobel_y = np.array([[-1, -2, -1],
                    [ 0,  0,  0],
                    [ 1,  2,  1]])

# Compute FFT of image
F = fft2(image)

# FFT of centered Sobel filters (use the provided function)
H_x = center_embed_kernel(sobel_x, image.shape)
H_y = center_embed_kernel(sobel_y, image.shape)

# Multiply in the frequency domain
G_x = F * H_x
G_y = F * H_y

# Back to the spatial domain
grad_x = ifft2(G_x).real
grad_y = ifft2(G_y).real
magnitude = np.sqrt(grad_x**2 + grad_y**2)

print("grad_x range  :", round(float(grad_x.min()), 3), "to", round(float(grad_x.max()),
3))
print("grad_y range  :", round(float(grad_y.min()), 3), "to", round(float(grad_y.max()),
3))
print("magnitude max :", round(float(magnitude.max()), 3))
```

```
grad_x range  : -3.18 to 3.303
grad_y range  : -2.979 to 2.665
magnitude max : 3.448
```

Taking `.real` is justified, not a shortcut

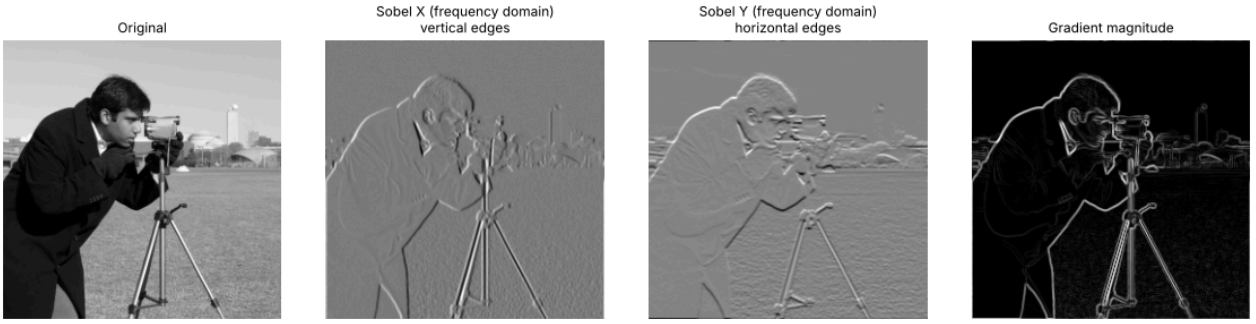
The inverse FFT returns a complex array. Discarding the imaginary part is only legitimate if it is already negligible — which it is, because the input is real and the filter is built from a real kernel. The residue is pure floating-point rounding.

In [14]:
```
print("largest imaginary component in the result:",
      f"{np.abs(ifft2(G_x).imag).max():.2e}", " <- floating-point noise, not signal")
print("largest real component                   :",
      f"{np.abs(ifft2(G_x).real).max():.4f}")
```

```
largest imaginary component in the result: 7.66e-16  <- floating-point noise, not signal
largest real component                   : 3.3026
```

Plot

In [15]:
```
show([image, grad_x, grad_y, magnitude],
     ["Original", "Sobel X (frequency domain)\nvertical edges",
      "Sobel Y (frequency domain)\nhorizontal edges",
      "Gradient magnitude"],
     cols=4, size=3.7)
```



The two directional results are visibly different, which is the point of the operator. The camera tripod and the vertical edges of the buildings appear in the X result; the horizon and the top of the coat appear in the Y result. Neither alone finds all the edges, but the magnitude does.

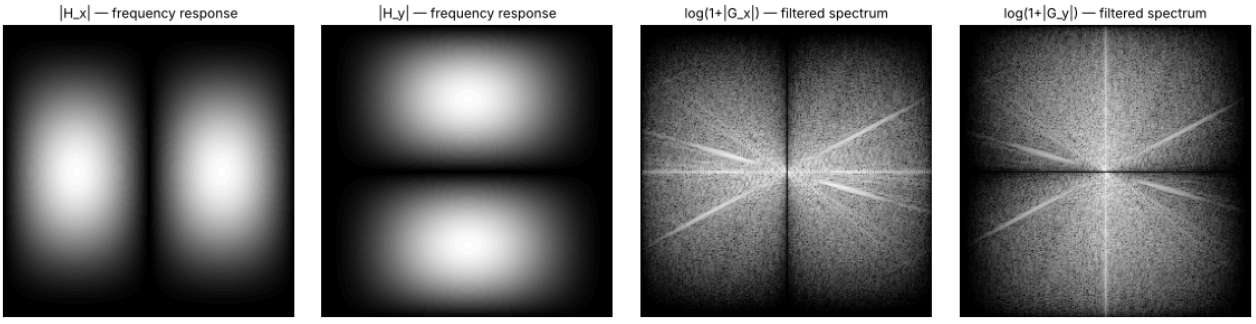
The frequency response of the Sobel filters

Plotting `|H|` shows directly why each filter is directional. `H_x` passes frequencies that vary along the horizontal axis and blocks the vertical ones; `H_y` is its transpose.

In [16]:
```
show([fftshift(np.abs(H_x)), fftshift(np.abs(H_y)),
      np.log(1 + np.abs(fftshift(G_x))), np.log(1 + np.abs(fftshift(G_y)))],
     ["|H_x| — frequency response", "|H_y| — frequency response",
      "log(1+|G_x|) — filtered spectrum", "log(1+|G_y|) — filtered spectrum"],
     cols=4, size=3.7)

print("H_x at the DC term:", f"{abs(H_x[0, 0]):.2e}",
      " <- zero: the Sobel kernel sums to 0, so flat regions give nothing")
print("H_y at the DC term:", f"{abs(H_y[0, 0]):.2e}")
```

```
H_x at the DC term: 0.00e+00  <- zero: the Sobel kernel sums to 0, so flat regions give nothing
H_y at the DC term: 0.00e+00
```



Both responses are zero at the centre and brightest towards the edges along one axis — a

directional high-pass filter, which is what an oriented edge detector is. The fact that the DC response is exactly zero matches the spatial fact from Lab 5 that the Sobel kernel's weights sum to zero.

Verification against the spatial domain

The whole point of the convolution theorem is that the two routes agree. They do, to about `1e-15`.

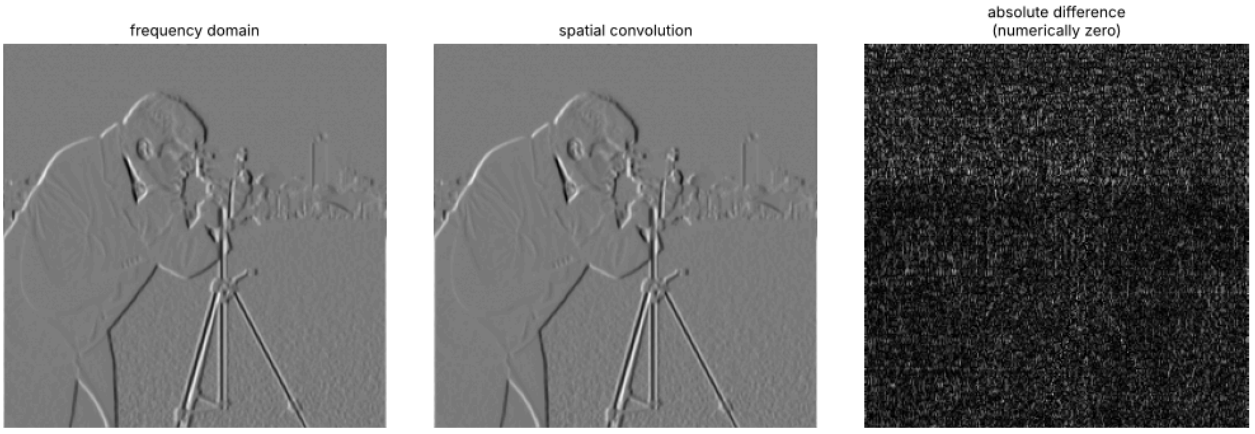
In [17]:
```
spatial_x = ndimage.convolve(image, sobel_x.astype(float), mode='wrap')
spatial_y = ndimage.convolve(image, sobel_y.astype(float), mode='wrap')

print("Sobel X — frequency vs spatial:", f"{np.abs(grad_x - spatial_x).max():.2e}")
print("Sobel Y — frequency vs spatial:", f"{np.abs(grad_y - spatial_y).max():.2e}")
print()
print("Both are zero to machine precision, which confirms the convolution theorem")
print("and that the kernel was embedded and shifted correctly.")

show([grad_x, spatial_x, np.abs(grad_x - spatial_x)],
     ["frequency domain", "spatial convolution", "absolute difference\n(numerically
zero)"],
     cols=3, size=4.4)
```

```
Sobel X — frequency vs spatial: 1.49e-15
Sobel Y — frequency vs spatial: 2.84e-15

Both are zero to machine precision, which confirms the convolution theorem
and that the kernel was embedded and shifted correctly.
```



Thresholding the gradient — an edge map

A practical use of the magnitude: thresholding it with Otsu's method, as in Lab 4, turns it into a binary edge map.

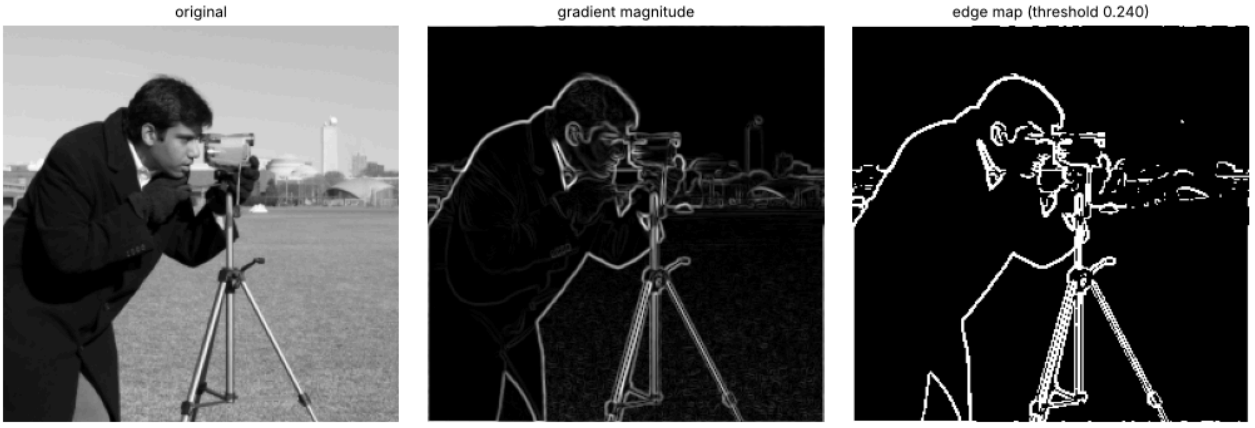
In [18]:
```
from skimage.filters import threshold_otsu

mag_norm = magnitude / magnitude.max()
t = threshold_otsu(mag_norm)
edges = mag_norm > t

print("Otsu threshold on the normalised magnitude:", round(float(t), 4))
print("pixels marked as edge:", f"{100 * edges.mean():.1f}%")

show([image, mag_norm, edges],
     ["original", "gradient magnitude", f"edge map (threshold {t:.3f})"],
     cols=3, size=4.4)
```

```
Otsu threshold on the normalised magnitude: 0.2403
pixels marked as edge: 7.5%
```



Summary figure

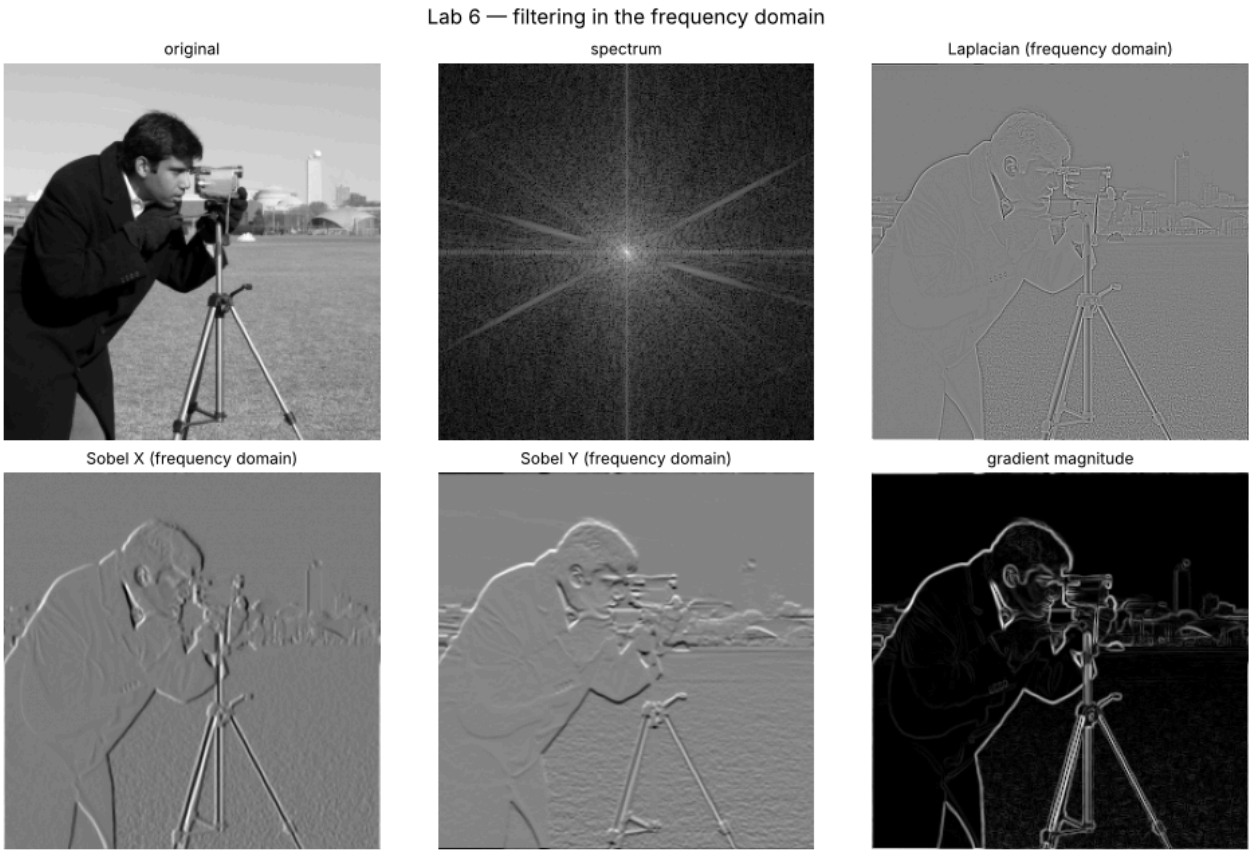
In [19]:

```
summary = [(image, "original"),
           (np.log(1 + np.abs(fftshift(F))), "spectrum"),
           (laplacian_image, "Laplacian (frequency domain)"),
           (grad_x, "Sobel X (frequency domain)"),
           (grad_y, "Sobel Y (frequency domain)"),
           (magnitude, "gradient magnitude")]

show([s[0] for s in summary], [s[1] for s in summary], cols=3, size=4.2,
     suptitle="Lab 6 — filtering in the frequency domain")

fig, axes = plt.subplots(2, 3, figsize=(13, 8.5))
for ax, (im, t_) in zip(axes.flat, summary):
    ax.imshow(im, cmap='gray')
    ax.set_title(t_, fontsize=11)
    ax.axis('off')
plt.tight_layout()
plt.savefig('output/lab6_summary.png', dpi=110, bbox_inches='tight')
plt.close()
print("Saved: output/lab6_summary.png")
```

```
Saved: output/lab6_summary.png
```



Conclusion

The frequency domain is the same image described differently — as a sum of waves rather than a grid of pixels. What makes it useful is that image content separates by frequency: low frequencies hold the broad shapes and overall brightness, high frequencies hold the edges and texture. Splitting the spectrum with a circular mask and reconstructing each half separately demonstrated this directly, producing a blurred image from the low frequencies and an edge-only image from the high ones.

Filtering becomes multiplication. The convolution theorem says that convolving with a kernel in the spatial domain is the same as multiplying by that kernel's transform in the frequency domain, and both the Laplacian and the Sobel results matched their spatial equivalents to about `1e-15`. The Laplacian filter `-4pi^2(u^2+v^2)` is zero at the origin and grows outward, which is a high-pass filter — the same conclusion Lab 5 reached from the fact that the spatial kernel sums to zero.

Three details decided whether the code was right.

`fftfreq` *is not centred*, despite the comment in the manual, and that is correct: its output order matches `fft2`'s, so the element-by-element multiplication lines up. `fftshift` exists for display and must never be applied to an array that is about to be used in a calculation.

`ifftshift` *before* `fft2` in `center_embed_kernel` moves the kernel's centre to index `[0,0]`, which is where the transform expects the origin. Omitting it produced an image translated by half the frame in each direction — verified by comparing against the correct result rolled by `(M//2, N//2)`, which matched to machine precision. This is the dangerous kind of error, since the output still looks like a plausible edge map.

The frequency product is convolution, not correlation. Lab 5 noted that `cv2.filter2D` performs correlation without flipping the kernel. For the symmetric kernels there the distinction was invisible;

the Sobel kernels are antisymmetric, so the two differ substantially — `1e-15` against the convolution and `6.6` against the correlation of the same image.

Finally, the FFT treats the image as periodic, so frequency-domain filtering wraps around at the edges. The spatial comparisons used `mode='wrap'` for exactly that reason; the disagreement between border modes was confined to the border pixels, as expected.

Resources

- NumPy FFT documentation: https://numpy.org/doc/stable/reference/routines.fft.html
- scikit-image sample data: https://scikit-image.org/docs/stable/auto_examples/data/plot_general.html